

Government College University Faisalabad

Department of Software Engineering
Bachelor of Science in Software Engineering

AgriGuard

Intelligent Crop Disease Detection and Smart Water Management System Software Requirements Specification (SRS)

Document Info	
Version	1.0
Date	15 January 2026
Supervisor	Dr. Zubair Nabi - Lab Engineer
Session	2022 - 2026

Team Members	
Arslan Ahmed	2022-GCUF-02591 mearslanahmed@gmail.com
Amna Ikram	2022-GCUF-02602 amnaikram822@gmail.com

1. Introduction

1.1 Purpose

This document defines the requirements for AgriGuard, a mobile system for crop disease detection and irrigation control. It is written for the development team, project supervisor, and academic evaluators. The goal is to establish an agreed baseline before development begins so that scope decisions and trade-offs are visible from the start.

1.2 Project Overview

AgriGuard is designed to help smallholder farmers in Pakistan to identify crop diseases early and manage irrigation without needing specialist knowledge. The system works in two modes. In the fixed plant setup, an ESP32-CAM module mounted above a tomato plant captures leaf images automatically and sends them for analysis without any input from the farmer. In the field, a farmer can also take a manual photo using the mobile app and get the same analysis on the spot.

Either way, the app shows the detected disease and what to do about it: which pesticide to use, at what dosage, and how often to apply it. The same app also lets the farmer turn the irrigation pump on or off remotely.

The system combines a machine learning model, a REST prediction API, a mobile application, a Node.js backend, and IoT hardware. Each component is developed and tested independently before being integrated.

1.3 Scope

Version 1.0 covers:

- Automatic leaf image capture via an ESP32-CAM module mounted on a fixed plant setup
- Manual leaf image capture via the farmer's phone camera, for field use
- Disease detection across tomato, potato, and pepper plants - 15 classes in total
- Pesticide recommendations per disease, including dosage, spray schedule, mixing ratio, and safety notes
- Remote water pump control through the mobile app
- Scan history stored per farmer account
- Admin access to update pesticide records without touching any code

Out of scope for version 1.0:

- Urdu language support, deferred to a future release
- Predictive disease forecasting based on weather or historical data
- Automated irrigation scheduling without manual input
- Multi-plant or multi-field monitoring

1.4 Definitions and Abbreviations

Term	Definition
SRS	Software Requirements Specification
ML	Machine Learning
API	Application Programming Interface
MobileNetV2	A lightweight convolutional neural network architecture designed for mobile and embedded image classification
PlantVillage	A publicly available dataset of labelled plant leaf images, used for training the disease detection model
ESP32-CAM	A low-cost Wi-Fi microcontroller module with an integrated camera. Used here to capture plant images automatically.
Expo	An open-source toolchain for building cross-platform React Native apps without native build tools
Flask	A Python web framework used to host the ML prediction endpoint
Node.js	A JavaScript runtime used for the application backend server
Express.js	A minimal Node.js web framework used to build the REST API
MongoDB	A NoSQL document database used to store user, scan, and pesticide data

Relay Module	An electronic switch that lets the ESP32 or microcontroller control a high-voltage device such as a water pump
JWT	JSON Web Token, used to manage authenticated sessions between the app and backend
REST	Representational State Transfer, the architectural style used for all HTTP APIs in this system

1.5 References

- AgriGuard FYP Proposal, Government College University Faisalabad, December 2025
- PlantVillage Dataset, available on Kaggle (emmarex/plantdisease)
- TensorFlow and Keras documentation - tensorflow.org
- React Native and Expo documentation - docs.expo.dev
- ESP32-CAM documentation - espressif.com
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications

2. Overall Description

2.1 System Components

AgriGuard has five components:

Component	Technology	Role
ML Model	MobileNetV2 / TensorFlow Keras	Classifies a leaf image into one of 15 disease categories
Prediction API	Flask / Python 3.11	Wraps the model behind an HTTP endpoint. Accepts an image, returns a JSON prediction.
Mobile App	React Native / Expo	The farmer-facing interface. Handles scanning, results, history, and pump control.
Backend Server	Node.js / Express / MongoDB	Manages user accounts, scan history, pesticide data, and pump commands.
Hardware Unit	ESP32-CAM + Water pump + Relay module	Captures leaf images automatically from the fixed plant and controls irrigation.

2.2 Image Input Methods

The system supports two ways to capture a leaf image for analysis:

Method	How It Works	Use Case
Automatic (ESP32-CAM)	The ESP32-CAM is mounted above the test plant. It captures an image on a schedule or on demand and sends it directly to the Flask prediction API over Wi-Fi. No farmer action required.	Fixed demo plant setup. Shows autonomous monitoring.

Manual Camera)	(Phone	The farmer opens the app, takes a photo of a leaf using the phone camera or picks one from the gallery, and submits it for analysis.	Field use. Lets the farmer scan any plant anywhere.
----------------	--------	--	---

2.3 User Roles

Role	Who They Are	Access
Farmer	A smallholder farmer with basic smartphone literacy. The primary user.	Scan leaves, view results, get pesticide advice, control pump, review scan history
Admin	A designated manager, typically from the development team. One account, provisioned at deployment.	View all 15 disease entries, edit pesticide recommendations for any disease class

2.4 Operating Environment

- Mobile app: Android 8.0 and above, iOS 13 and above, built with React Native via Expo
- Prediction API: Linux server running Python 3.11 and Flask
- Backend: Node.js 18 or later with MongoDB
- Hardware: ESP32-CAM module with a relay-controlled water pump, connected to the same Wi-Fi network as the server
- Network: Wi-Fi required for the ESP32-CAM and the mobile app to communicate with the backend

2.5 Assumptions

- The farmer has a working smartphone and an internet connection when using the app in the field
- The ESP32-CAM and the backend server are on the same local network during the demo
- Leaf images are taken in reasonable lighting; dark or blurry images will reduce detection accuracy
- The admin manages pesticide data manually through the app; no external data feed is connected
- One admin account is set up at deployment; there is no self-registration for admin users

2.6 Constraints

- The ML model only recognises the 15 disease classes it was trained on; submitting an unsupported crop will give unreliable results
- The system is not a replacement for a qualified agricultural advisor
- Hardware integration is scoped to a single tomato plant for the FYP demo
- Urdu language support is not included in version 1.0

3. Functional Requirements

3.1 Authentication

ID	Requirement	Priority
FR-01	Farmers can register with their name, phone number/ Email, and a password.	High
FR-02	Registered farmers can log in with their phone number/ Email and password.	High
FR-03	On successful login, the server issues a JWT token used for all authenticated requests.	High
FR-04	Admin login uses a separate screen with fixed admin credentials. Admins cannot self-register.	High
FR-05	A user can log out; the session token is invalidated.	Medium

3.2 Automatic Image Capture (ESP32-CAM)

ID	Requirement	Priority
FR-06	The ESP32-CAM module captures a leaf image from the fixed plant setup and sends it to the Flask /predict endpoint via HTTP POST over Wi-Fi.	High
FR-07	The ESP32-CAM can be triggered on a time interval or on demand from the mobile app.	Medium
FR-08	The prediction result from the ESP32-CAM capture is forwarded to the backend and displayed in the farmer's app automatically.	High

3.3 Manual Image Capture (Phone Camera)

ID	Requirement	Priority
FR-09	The farmer can take a photo using the phone camera from within the app.	High
FR-10	The farmer can pick an existing image from their phone gallery.	Medium
FR-11	The app sends the selected image to the Flask /predict endpoint via HTTP POST.	High

3.4 Disease Detection and Results

ID	Requirement	Priority
FR-12	The Flask API resizes the received image to 224x224 pixels and normalises pixel values to the 0-1 range before running inference.	High

FR-13	The API returns a JSON object with crop name, disease name, confidence percentage, and a Boolean healthy flag.	High
FR-14	The app displays the result to the farmer in plain English with no raw class names or technical identifiers.	High
FR-15	A confidence score is shown with the result. Results below 60% confidence show a low-confidence warning.	Medium

3.5 Pesticide Advisory

ID	Requirement	Priority
FR-16	After a disease is identified, the app retrieves the matching pesticide record from the backend.	High
FR-17	The pesticide record includes pesticide name, dosage, spray interval, water mixing ratio, and safety instructions.	High
FR-18	If the crop is healthy, the app shows a positive status message instead of a pesticide recommendation.	High
FR-19	Only admin users can edit pesticide records. Farmers have read-only access.	High

3.6 Scan History

ID	Requirement	Priority
FR-20	Each completed scan is saved to the database, linked to the farmer's account, with crop, disease, confidence, source (auto or manual), and timestamp.	High
FR-21	The farmer can view their scan history, newest first.	Medium
FR-22	Each history entry shows the result, confidence, source, and date of the scan.	Medium

3.7 Water Pump Control

ID	Requirement	Priority
FR-23	The app shows the current pump state (on or off) on the pump control screen.	High
FR-24	The farmer can toggle the pump on or off with a single tap.	High
FR-25	The backend forwards the control command to the ESP32 or relay controller.	High
FR-26	The app confirms the new pump state after the command is processed. A failed command shows an error.	Medium

3.8 Admin Panel

ID	Requirement	Priority
FR-27	Admin users access a separate login screen from the farmer login.	High
FR-28	After login, the admin sees all 15 disease entries and their current pesticide recommendations.	High
FR-29	The admin can edit the pesticide record for any disease and save changes.	High
FR-30	Updated records are served to the farmer app immediately, with no redeployment needed.	High

4. Non-Functional Requirements

4.1 Performance

- The prediction API should return a result within 5 seconds of receiving an image under normal Wi-Fi conditions.
- The app home screen should load within 3 seconds on a mid-range Android device.
- The backend should handle at least 10 concurrent requests without noticeable slowdown during the demo.

4.2 Accuracy

- The disease detection model must achieve at least 90% accuracy on a held-out test set from the PlantVillage dataset before the system is considered ready for demo.
- Predictions below 60% confidence are flagged in the UI so the farmer knows the result may not be reliable.

4.3 Usability

- A farmer with no prior app experience should be able to complete a manual scan within 3 taps of opening the app.
- No raw model output, class indices, or technical jargon is shown to the farmer at any point.
- Error messages are written in plain English and explain what the farmer should do next, not just what went wrong.

4.4 Reliability

- If the prediction API is unreachable, the app shows a clear error message rather than hanging or crashing.
- Scan records are written to the database before the result screen is shown; a write failure is surfaced to the user.
- Rapid taps on the pump toggle do not queue duplicate commands to the hardware.

4.5 Security

- All backend endpoints except /auth/register and /auth/login require a valid JWT. Requests without one get a 401 response.

- Passwords are stored as bcrypt hashes. Plaintext passwords are never written to the database or server logs.
- Admin endpoints reject requests from farmer-role tokens even if the token itself is valid.
- HTTPS is used for any deployment outside the local network.

4.6 Maintainability

- Pesticide data is stored in MongoDB. An admin can update it through the app with no code changes required.
- The ML model is a standalone file. Swapping it for a retrained version only requires replacing the file on the server, not changing any API code.
- Each system component has its own folder in the repository: ml_model, flask_api, backend, mobile_app, hardware.

4.7 Portability

- The mobile app runs on Android and iOS from a single codebase via React Native.
- The Flask API and Node.js backend run on any Linux server without platform-specific dependencies.
- The ESP32-CAM firmware is written to be flashable on any ESP32-CAM board without hardware modifications.

5. System Architecture

5.1 Component Stack

Layer	Component	Technology
Presentation	Mobile Application	React Native / Expo
ML Inference	Prediction API	Flask / Python 3.11 / TensorFlow / Keras
Business Logic	Backend Server	Node.js / Express.js
Data	Database	MongoDB
Hardware	IoT Unit	ESP32-CAM + Relay module + Water pump

5.2 Data Flow - Automatic Capture (ESP32-CAM)

1. ESP32-CAM captures a leaf image from the mounted plant on a schedule or on demand.
2. ESP32-CAM sends the image directly to the Flask /predict endpoint via HTTP POST over Wi-Fi.
3. Flask resizes the image to 224x224, normalises pixel values, runs the MobileNetV2 model, and returns a JSON prediction.
4. The result is forwarded to the Node.js backend and stored in the scans collection.
5. The farmer's app polls or receives a push notification and displays the new result.

5.3 Data Flow - Manual Capture (Phone Camera)

6. Farmer opens the Scan screen, takes a photo or picks one from the gallery.
7. The app sends the image to Flask /predict via HTTP POST.
8. Flask returns crop, disease, confidence, and healthy flag as JSON.
9. The app posts the result to Node.js backend to save in scan history.
10. The app fetches the matching pesticide record from /api/diseases.
11. The results screen shows the diagnosis and pesticide advisory together.

5.4 Planned API Endpoints

Method	Endpoint	Description	Auth
POST	/predict	Accept image, return disease prediction	None
POST	/api/auth/register	Register a new farmer account	None
POST	/api/auth/login	Log in, receive JWT token	None
GET	/api/history	Get the logged-in farmer's scan history	Farmer JWT
POST	/api/history	Save a new scan result	Farmer JWT
GET	/api/diseases	Get all disease and pesticide records	Farmer JWT
PUT	/api/diseases/:id	Update a pesticide recommendation	Admin JWT
GET	/api/pump/status	Get current pump state	Farmer JWT
POST	/api/pump/control	Send pump on or off command	Farmer JWT

6. Data Requirements

6.1 Disease Classes

The system will support the following 15 classes from the PlantVillage dataset:

ID	Class	Crop	Type
0	Pepper Bell Bacterial Spot	Pepper	Disease
1	Pepper Bell Healthy	Pepper	Healthy
2	Potato Early Blight	Potato	Disease
3	Potato Late Blight	Potato	Disease
4	Potato Healthy	Potato	Healthy
5	Tomato Bacterial Spot	Tomato	Disease
6	Tomato Early Blight	Tomato	Disease

7	Tomato Late Blight	Tomato	Disease
8	Tomato Leaf Mold	Tomato	Disease
9	Tomato Septoria Leaf Spot	Tomato	Disease
10	Tomato Spider Mites	Tomato	Disease
11	Tomato Target Spot	Tomato	Disease
12	Tomato Yellow Leaf Curl Virus	Tomato	Disease
13	Tomato Mosaic Virus	Tomato	Disease
14	Tomato Healthy	Tomato	Healthy

6.2 Database Collections

Collection	Key Fields	Purpose
users	name, phone, passwordHash, role (farmer / admin), createdAt	Farmer and admin accounts
scans	userId, crop, disease, confidence, isHealthy, source (auto / manual), createdAt	Scan history per farmer. Source field distinguishes ESP32-CAM captures from manual phone captures.
diseases	classId, className, crop, disease, pesticide, dosage, sprayInterval, mixingRatio, safetyNotes	Pesticide recommendations, editable by admin through the app
pumpStatus	status (on / off), updatedAt, updatedBy	Tracks the current pump state and who last changed it

7. Interface Requirements

7.1 Mobile App Screens

- Login screen for farmers with a link to registration; separate admin login screen
- Home screen with quick navigation to Scan, History, and Pump Control
- Scan screen with two options: take a photo now, or pick from the gallery
- Results screen showing disease name, confidence score, and the full pesticide advisory in one view
- History screen listing past scans newest first, with source indicator (auto or manual)
- Pump Control screen with a clear on/off toggle and a live status indicator
- Admin screen showing all 15 disease entries with editable pesticide fields

7.2 Hardware Interface

- ESP32-CAM module mounted above the tomato plant, connected to the local Wi-Fi network
- The ESP32-CAM sends captured images to Flask /predict via HTTP POST
- A water pump connected to the ESP32 or a separate microcontroller through a relay module

- Pump control commands arrive from the Node.js backend as JSON: { "action": "on" } or { "action": "off" }
- The hardware unit sends a confirmation response after executing a pump command

7.3 ML Model Interface

- Input: JPEG or PNG image at any resolution. The API handles resizing internally.
- Output: JSON with fields: crop, disease, class_name, confidence (percentage), is_healthy, success
- The model file is loaded once at Flask server startup and kept in memory. It is not reloaded per request.

8. Risk Assessment

Risk	Impact	Likelihood	Mitigation
Model accuracy drops on real plant photos compared to the clean dataset images	High	Medium	Test on actual tomato leaf photos early in development. Add a UI note advising good lighting and a close-up shot for manual captures.
ESP32-CAM integration takes longer than expected	Medium	Medium	Build and test the full software stack first. The ESP32-CAM connects at the end and does not block the core demo.
Scope grows mid-development as new ideas come up	Medium	Medium	New ideas go on a backlog. Only the requirements in this document are in scope for version 1.0.
App is unusable without a network connection	Medium	Low	Show a clear offline error immediately. Never let the app hang silently on a failed network request.
Not enough time to complete all modules before submission	High	Medium	The scan-to-result flow is the non-negotiable core. Scan history, admin panel, and hardware are secondary if time runs short.

9. Future Enhancements

These features are not in scope for version 1.0 but are worth building in later iterations:

- Urdu language support with right-to-left layout, for farmers who are not comfortable with English
- On-device inference using a TensorFlow Lite model, so the app works without an internet connection
- Automated irrigation scheduling based on a timer or soil moisture sensor readings
- Support for additional crops beyond tomato, potato, and pepper
- Push notifications to remind farmers when the next spray interval is due
- Weather API integration to adjust irrigation recommendations based on local forecast
- Multi-plant monitoring with multiple ESP32-CAM units across a field

Supervisor's Remarks

Remarks:

Signature:		Date:
Dr. Zubair Nabi Lab Engineer, Dept. of Software Engineering Government College University Faisalabad		